

Shared Dynamics Contracts Specification

H0: Analysis Subspaces, Semantic Signatures, Deep Immutability, and Strict Validation

mdstats

2026-07-22

Contents

Purpose and status	1
Scientific boundary	2
Borrowed physical relations	2
mdstats-specific design	2
Public and internal API	2
Analysis-subspace contract	3
Resolver	3
Result type	3
Projection of stored tensors	3
Semantic input signature	4
Result type	4
Trajectory fingerprint	4
Signature construction	4
Compatibility	5
Deep-immutability contract	5
Strict option validation	6
Integer controls	6
Boolean controls	6
Finite real controls	6
Green-Kubo and reconstruction integration	6
Running diffusion	6
VACF-to-MSD reconstruction	6
Plateau and comparison integration	7
Explicit plateau estimator	7
MSD/VACF diffusion comparison	7
Spectrum and VDOS propagation	7
Source mapping	8
Required focused tests	8
Acceptance condition	8
References	8

Purpose and status

This document specifies the H0 dynamics-contract hardening implemented in

`mdstats/analysis/_dynamics_common.py`

and integrated into

`mdstats/analysis/_velocity_common.py`

`mdstats/analysis/vacf.py`

`mdstats/analysis/msd.py`

`mdstats/analysis/vacf_transport.py`

`mdstats/analysis/diffusion.py`

`mdstats/analysis/velocity_spectrum.py`

H0 introduces no new physical estimator. It repairs the common boundary used by existing VACF, MSD, Green-Kubo, reconstruction, diffusion-comparison, spectrum, and VDOS calculations. The stage is complete when:

1. a physical analysis subspace is explicit and its rank is the only dimensional divisor;
2. cross-module comparisons use complete semantic input signatures;
3. public dynamics results are deeply immutable;
4. integer, boolean, and finite-scalar options follow one strict validation policy; and
5. the existing arithmetic-mean plateau estimator accepts only uniformly spaced selected samples.

The first new displacement observable remains D0 and is outside this specification.

Scientific boundary

Borrowed physical relations

The Green-Kubo self-diffusion relation is borrowed from Green [1] and Kubo [2]. For an orthonormal basis $B \in \mathbb{R}^{d \times 3}$ with rows $\mathbf{b}_a$, define the projector

$$P = B^T B.$$

For the Cartesian velocity-correlation tensor $C(t)$, the correlation restricted to the selected physical subspace is

$$C_B(t) = [B C(t) B^T] = [P C(t)].$$

The corresponding self-diffusion estimate is

$$D_B(t) = \frac{1}{d} \int_0^t C_B(\tau) d\tau, \quad d = \text{rank}(B).$$

The Einstein long-time relation used by the comparison layer is borrowed from Einstein [3]:

$$\left\langle \left\| B \Delta \mathbf{r}(t) \right\|^2 \right\rangle \sim 2d D_B t.$$

H0 does not modify these relations. It prevents an unrelated integer from being used as d without first selecting the corresponding physical subspace.

mdstats-specific design

The following are package design rather than borrowed estimators:

- the AnalysisSubspace representation and axis-label compatibility layer;
- projector-based equivalence of rotated or sign-flipped orthonormal bases;
- DynamicsInputSignature and its fail-closed comparison policy;
- deterministic trajectory-content fingerprinting;
- recursive metadata freezing and owned read-only result arrays;
- strict rejection of booleans where integer controls are required; and
- the uniform-grid admission rule for the current plateau arithmetic mean.

Public and internal API

H0 exports the following public contract objects:

```
from mdstats import (
    AnalysisSubspace,
    DynamicsInputSignature,
    resolve_analysis_subspace,
)
```

The remaining helpers are internal:

```
owned_readonly_array(...)
freeze_nested(...)
```

```

freeze_mapping(...)
require_bool(...)
require_positive_int(...)
require_nonnegative_int(...)
require_finite_real(...)
resolve_subspace_with_legacy_options(...)
project_trace_from_result(...)
trajectory_fingerprint(...)
build_dynamics_signature(...)

```

Analysis-subspace contract

Resolver

```

def resolve_analysis_subspace(
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
) -> AnalysisSubspace:
    ...

```

Exactly one of `axes` and `projection_basis` may be supplied. With neither, the resolver returns the full Cartesian basis.

For `axes`:

- the input is a nonempty sequence, not a string;
- entries are unique and belong to "x", "y", and "z";
- user ordering is retained; and
- the basis rows are the corresponding Cartesian unit vectors.

For `projection_basis`:

- a one-dimensional vector is promoted to shape (1, 3);
- the accepted shape is (d, 3) with $d \in \{1, 2, 3\}$;
- all entries are finite; and
- rows satisfy

$$BB^T = I_d$$

within relative tolerance 10^{-10} and absolute tolerance 10^{-12} .

The implementation rejects malformed, rank-deficient, scaled, or nonorthogonal bases. It does not silently orthonormalize user input because doing so would change the requested observable.

Result type

```

@dataclass(frozen=True, slots=True, eq=False)
class AnalysisSubspace:
    projection_basis: NDArray[np.float64] # (d, 3), owned/read-only
    labels: tuple[Literal["x", "y", "z"], ...] | None
    rank: int

```

projector returns the read-only matrix $B^T B$. `same_physical_subspace(other)` compares projectors rather than basis rows. Thus, basis-row rotations, permutations, and sign changes that span the same subspace are physically equivalent.

`component_label` is retained only for compatibility:

- a single labeled Cartesian axis returns "x", "y", or "z";
- every other subspace returns "scalar".

Projection of stored tensors

For canonical axis subsets, the projected trace may be assembled from stored Cartesian diagonal components. A general rotated basis requires the full tensor:

$$C_B(t) = \sum_{a=1}^d \mathbf{b}_a^T C(t) \mathbf{b}_a.$$

If a rotated projection is requested and the source result does not retain its full tensor, the consumer raises. It must not discard the off-diagonal terms.

Semantic input signature

Result type

```
@dataclass(frozen=True, slots=True, eq=False)
class DynamicsInputSignature:
    source_format: str | None
    source_files: tuple[str, ...]
    trajectory_fingerprint: str

    frame_indices: tuple[int, ...] | None
    frame_times_ps: NDArray[np.float64]
    n_frames: int
    sample_spacing_ps: float | None

    atom_indices: NDArray[np.int64]
    coordinate_mode: str
    reference_cell_mode: str | None
    reference_cell: NDArray[np.float64] | None

    drift_mode: str | None
    drift_atom_indices: NDArray[np.int64] | None
    velocity_source: str | None

    projection_basis: NDArray[np.float64]
    projection_labels: tuple[Literal["x", "y", "z"], ...] | None
```

Every array is owned and read-only. Atom indices are unique. The frame-time array must match `n_frames`. A stored sample spacing is finite and positive; it is `None` for a single frame or a nonuniform grid.

Trajectory fingerprint

`trajectory_fingerprint(collection)` computes a deterministic SHA-256 digest of normalized trajectory content relevant to current dynamics calculations:

- frame IDs;
- steps;
- times;
- atomic numbers and masses;
- periodic-boundary flags;
- cells and Cartesian cell origins;
- fractional positions; and
- velocities when present.

Each field contributes an explicit label, canonical little-endian dtype, shape, and contiguous byte representation. The digest distinguishes different slices or content even when the source filename is unchanged. It is an identity check, not a cryptographic security claim.

Signature construction

```
def build_dynamics_signature(
    collection,
    *,
    atom_indices,
    coordinate_mode,
```

```

        reference_cell_mode,
        reference_cell,
        drift_mode,
        drift_atom_indices,
        velocity_source,
        subspace=None,
    ) -> DynamicsInputSignature:
        ...

```

The helper reads normalized source provenance from the frame collection and records the exact analyzed frame sequence, measured atoms, drift-reference atoms, coordinate/reference-cell semantics, velocity source, and observable subspace.

Velocity preparation creates the source 3D signature. A projected transport or comparison result uses `signature.with_subspace(subspace)` without changing any other identity field.

Compatibility

`mismatch_fields(other)` compares:

- source format and filenames;
- trajectory fingerprint;
- frame IDs and frame times;
- frame count and sample spacing;
- measured atoms;
- coordinate and reference-cell semantics;
- exact drift mode and drift-reference atoms;
- velocity source; and
- the physical projection subspace.

Arrays use exact equality because they identify normalized inputs rather than approximate measured outcomes. Subspaces use projector equivalence.

A cross-module comparison fails closed if either result lacks a complete signature. Signed source-result constructors also verify that signature atom, drift, coordinate, reference-cell, and source-subspace fields agree with their own explicit fields. Direct construction of an unsigned legacy result remains possible for low-level compatibility and isolated tests, but that result is not silently accepted as comparable scientific provenance.

Deep-immutability contract

A frozen dataclass alone does not protect contained NumPy arrays or dictionaries. H0 therefore requires:

1. every public result array is copied into owned C-contiguous storage;
2. `array.flags.writeable` is false;
3. metadata mappings become `MappingProxyType` instances;
4. nested mappings are recursively frozen;
5. lists and tuples become tuples;
6. sets become frozensets; and
7. nested NumPy arrays are also copied and made read-only.

The input object may be modified after result construction without changing the result. Attempts to modify a result array, nested metadata array, metadata key, or nested sequence must raise.

The policy applies to public result types:

- `VACFResult`;
- `MSDResult`;
- `VACFDiffusionResult`;
- `VACFMSDResult`;
- `DiffusionEstimate`;
- `DiffusionComparisonResult`;
- `VelocitySpectrumResult`; and

- `VDOSResult`.

`VelocityInputBundle` is a private, short-lived preparation object. To avoid a full trajectory copy, its `velocities` member retains the normalized collection array. Its small selection, weighting, drift, and signature arrays are owned and read-only. No public result aliases that trajectory array.

Strict option validation

Integer controls

Positive integer controls reject booleans and accept Python or NumPy integer scalars. Relevant controls include:

- origin and lag strides;
- maximum lag counts;
- atom block sizes;
- minimum plateau points; and
- other public dynamics point-count controls migrated in H0.

Zero or negative values raise according to the option contract.

Boolean controls

Boolean switches require `bool` or `numpy.bool_`. Integer substitutes such as 0 and 1 raise. This applies to switches such as `tensor` and `per-atom output`.

Finite real controls

Shared finite-real validation rejects booleans, nonnumeric values, NaN, and infinity. Positive or nonnegative constraints are explicit at the call site.

Green-Kubo and reconstruction integration

Running diffusion

```
def integrate_vacf_to_diffusion(
    vacf: VACFResult,
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
    projection_basis: ArrayLike | None = None,
    dimensions: Literal[1, 2, 3] | None = None,
    component: Literal["scalar", "x", "y", "z"] = "scalar",
    maximum_time_ps: float | None = None,
    integration: Literal["trapezoid"] = "trapezoid",
) -> VACFDiffusionResult:
    ...
```

`axes` or `projection_basis` is authoritative. `dimensions` and `component` are compatibility adapters:

- default scalar means the full 3D basis;
- `component="x", "y", or "z"` selects that one axis;
- scalar `dimensions=3` remains accepted as an unambiguous consistency check;
- scalar `dimensions=1` or `2` without an explicit corresponding subspace raises;
- `dimensions` cannot disagree with an explicit subspace rank; and
- a Cartesian component cannot be combined with another explicit subspace.

The projected correlation is formed first and divided by the resolved rank only then. The source signature is propagated with the resolved subspace.

VACF-to-MSD reconstruction

```
def reconstruct_msd_from_vacf(
    vacf: VACFResult,
    *,
    axes: Sequence[Literal["x", "y", "z"]] | None = None,
```

```

projection_basis: ArrayLike | None = None,
component: Literal["scalar", "x", "y", "z"] = "scalar",
maximum_time_ps: float | None = None,
integration: Literal["trapezoid"] = "trapezoid",
) -> VACFMSDResult:
    ...

```

For projected correlation C_B , the reconstruction remains

$$M_B(t) = 2 \left[t \int_0^t C_B(\tau) d\tau - \int_0^t \tau C_B(\tau) d\tau \right].$$

No dimensional divisor is applied because the output is the total projected mean-square displacement, not a diffusion coefficient.

Plateau and comparison integration

Explicit plateau estimator

`estimate_diffusion_plateau()` retains the arithmetic mean of stored running values over a user-selected interval. H0 adds the admission rule

$$\Delta t_j = \Delta t$$

for all selected adjacent samples within documented floating-point tolerance. An irregular selected grid raises rather than weighting dense regions more heavily. A future time-weighted estimator must use a separately named method.

The result continues to report descriptive diagnostics only. Adjacent values of one running integral are serially correlated and are not treated as independent replicates for an inferential standard error.

MSD/VACF diffusion comparison

```

def compare_msd_vacf_diffusion(
    msd: MSDResult,
    vacf_diffusion: DiffusionEstimate,
    *,
    msd_fit_range_ps: tuple[float, float],
    dimensions: Literal[1, 2, 3] | None = None,
) -> DiffusionComparisonResult:
    ...

```

The comparison:

1. requires time-averaged laboratory-frame MSD;
2. requires complete signatures on both sides;
3. replaces the MSD signature subspace with the estimate subspace and compares every remaining semantic field;
4. projects the MSD tensor or Cartesian components onto that same subspace;
5. fits the selected projected MSD with an intercept; and
6. divides the slope by $2d$, where d is the stored subspace rank.

The optional `dimensions` argument is a deprecated consistency check. It cannot reinterpret the stored estimate.

Spectrum and VDOS propagation

VACF-transform spectra preserve the input VACF signature. Direct Welch spectra preserve the signature produced by shared velocity preparation. VDOS results preserve the input spectrum signature. These transformations alter the numerical representation, not the physical trajectory, selection, drift, or projection identity.

All spectrum and VDOS arrays and metadata follow the H0 deep-immutability policy.

Source mapping

Contract	Primary implementation
Subspace, signature, freezing, strict validators	<code>_dynamics_common.py</code>
Velocity-source signature construction	<code>_velocity_common.py</code>
Source signatures and immutable VACF results	<code>vacf.py</code>
Source signatures and immutable MSD results	<code>msd.py</code>
Projected Green-Kubo and reconstruction	<code>vacf_transport.py</code>
Plateau admission and fail-closed comparison	<code>diffusion.py</code>
Spectrum/VDOS signature propagation	<code>velocity_spectrum.py</code>

Required focused tests

H0 acceptance requires tests that:

1. preserve existing full-3D scalar Green-Kubo and Cartesian-component values;
2. distinguish anisotropic xy, xz, and full-3D traces;
3. verify a rotated projection with nonzero off-diagonal tensor terms;
4. reject a rotated projection when the full tensor is absent;
5. reject scalar dimensions=1 or 2 without an explicit subspace;
6. detect different drift-reference atoms under the same drift-mode label;
7. detect different frame times, cell origins, masses, or trajectory content under the same filename;
8. reject coordinate/reference-cell and velocity-source mismatches;
9. project the MSD with the same basis used by the VACF estimate;
10. reject unsigned legacy results in cross-module comparison;
11. propagate signatures through VACF spectrum and VDOS;
12. reject mutation of arrays and nested metadata;
13. reject booleans for integer controls and integers for boolean switches; and
14. reject an irregular selected plateau grid.

The H0-specific adversarial file is

`tests/test_dynamics_hardening.py`

Existing VACF, MSD, transport, diffusion, spectrum, VDOS, and plotting tests are also part of the focused regression gate.

Acceptance condition

H0 is accepted when:

- the new common module and public exports are present;
- all affected result schemas validate and freeze their contents;
- every computed MSD and VACF has a complete source signature;
- projected transport and comparisons derive dimensionality from the basis;
- legacy unambiguous 3D and Cartesian calls preserve numerical results;
- ambiguous legacy one-/two-dimensional scalar calls fail with a targeted error;
- the dedicated and existing focused test sets pass; and
- this specification, affected module specifications, architecture manual, changelog, and package version agree.

References

[1] M. S. Green, “Markoff Random Processes and the Statistical Mechanics of Time-Dependent Phenomena. II. Irreversible Processes in Fluids,” *Journal of Chemical Physics* **22**, 398-413 (1954). DOI: 10.1063/1.1740082.

- [2] R. Kubo, "Statistical-Mechanical Theory of Irreversible Processes. I. General Theory and Simple Applications to Magnetic and Conduction Problems," *Journal of the Physical Society of Japan* **12**, 570-586 (1957). DOI: 10.1143/JPSJ.12.570.
- [3] A. Einstein, "Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen," *Annalen der Physik* **322**, 549-560 (1905). DOI: 10.1002/andp.19053220806.